MODUL PRAKTIKUM 3

Batch Data Analytics + Visualization Layer

Big Data Dashboard (Microsoft Power BI / *Business Intelligence*)

Programming Language	: **Python**
Development Environment	: ***Linux Server Environment***
Code Editor	: **VS Code** (*Remote* ***WSL***)
Command Line Interface	: **Bash**
Distributed Data Processing Engine	: **PySpark**
Business Intelligence Platform	: **Power BI** / Tableau / Looker / Metabase / Apache Superset / Qlik Sense
Durasi	: **150 menit**

---

1. TUJUAN PRAKTIKUM

Mahasiswa mampu:

1. Memahami peran **Visualization Layer** dalam arsitektur Big Data.
2. Menggunakan **Power BI** untuk membuat dashboard analitik.
3. Menghubungkan dataset hasil pipeline Spark ke tools BI.
4. Membuat **KPI dashboard sederhana** dari dataset e-commerce.
5. Menyajikan insight bisnis dari data yang telah diproses.

2. KONTEKS INDUSTRI

Dalam industri data modern, pipeline data biasanya terdiri dari beberapa layer:

- **Data Source** → sumber data mentah
- **Processing Layer** → pemrosesan data (Spark)
- **Storage Layer** → penyimpanan (Parquet / Data Lake)
- **Analytics Layer** → agregasi dan KPI
- **Visualization Layer** → dashboard untuk pengambilan keputusan

Visualization Layer digunakan oleh:

- Manajer bisnis
- Tim operasional
- Eksekutif perusahaan

untuk memahami kondisi bisnis melalui **dashboard interaktif**.

Tools yang umum digunakan:

- Power BI
- Tableau
- Looker
- Metabase
- Apache Superset
- Qlik Sense

Pada praktikum ini kita menggunakan **Power BI Desktop**.

3. ARSITEKTUR PIPELINE

Pipeline yang digunakan dalam praktikum ini adalah:

```
Raw Dataset (CSV)
        ↓
Spark Processing
        ↓
Clean Data (Parquet)
        ↓
Analytics Layer
        ↓
Serving Layer (CSV)
        ↓
Power BI Dashboard
```

3.1 Konsep yang Digunakan

1. **Data Pipeline**

   Data pipeline adalah alur proses yang mengubah data mentah menjadi informasi yang siap digunakan.

   Contoh pipeline pada praktikum:

   ```
   CSV → Spark → Clean Data → KPI → Dashboard
   ```

2. **Analytics Layer**

   Analytics Layer adalah proses agregasi data untuk menghasilkan metrik bisnis seperti:

   - total revenue
   - top product
   - revenue per category

   Layer ini biasanya dibuat menggunakan:

   - Spark
   - SQL
   - Python

3. **Visualization Layer**

   Visualization Layer adalah tahap terakhir pipeline data yang menampilkan insight dalam bentuk visual seperti:

   - KPI card
   - bar chart
   - dashboard

   Tujuannya adalah membantu pengambil keputusan memahami data dengan cepat.

4. LANGKAH-LANGKAH PRAKTIKUM

Sekarang kita akan **lengkapi arsitektur sampai Visualization Layer**, supaya dapat melihat gambaran *end-to-end Big Data Technology*.

BAGIAN A — ANALYTICS LAYER

Tujuan: Membuat dataset yang benar-benar siap BI (*Business Intelligence*).

STEP 1 — Buat Script Analytics

Buat file baru:

`scripts/analytics_layer.py`

Isi dengan:

```
# ====================================
# ANALYTICS + SERVING LAYER
# ====================================

from pyspark.sql import SparkSession
from pyspark.sql.functions import sum as _sum, avg, desc
import os
import time

# ============================
# START TIMER
# ============================
start_time = time.time()

print("========================================")
print("        ANALYTICS LAYER STARTED        ")
print("========================================")

# ============================
# INIT SPARK
# ============================
spark = SparkSession.builder \
    .appName("AnalyticsLayer") \
    .master("local[*]") \
    .getOrCreate()

spark.sparkContext.setLogLevel("ERROR")

# ============================
# CREATE SERVING FOLDER
# ============================
if not os.path.exists("data/serving"):
    os.makedirs("data/serving")

# ============================
# LOAD CLEAN DATA (SILVER)
```

```
# ============================
print("Loading Clean Parquet Data...")
df_clean = spark.read.parquet("data/clean/parquet/")

total_records = df_clean.count()
print(f"Total Records: {total_records}")
print("----------------------------------------")

# ============================
# KPI 1: TOTAL REVENUE
# ============================
print("Calculating Total Revenue...")

total_revenue = df_clean.agg(
    _sum("total_amount").alias("total_revenue")
)

total_revenue.show()

# Save as CSV
total_revenue.write.mode("overwrite") \
    .option("header", True) \
    .csv("data/serving/total_revenue")

print("Total Revenue saved to data/serving/total_revenue")
print("----------------------------------------")

# ============================
# KPI 2: TOP 10 PRODUCTS
# ============================
print("Calculating Top 10 Products...")

top_products = df_clean.groupBy("product") \
    .agg(_sum("quantity").alias("total_quantity")) \
    .orderBy(desc("total_quantity")) \
    .limit(10)

top_products.show()

# Save as CSV
top_products.write.mode("overwrite") \
    .option("header", True) \
    .csv("data/serving/top_products")

print("Top Products saved to data/serving/top_products")
print("----------------------------------------")

# ============================
# KPI 3: REVENUE PER CATEGORY
# ============================
print("Calculating Revenue per Category...")
```

```
category_revenue = df_clean.groupBy("category") \
    .agg(_sum("total_amount").alias("category_revenue")) \
    .orderBy(desc("category_revenue"))

category_revenue.show()

# Save as CSV
category_revenue.write.mode("overwrite") \
    .option("header", True) \
    .csv("data/serving/category_revenue")

print("Category Revenue saved to data/serving/category_revenue")
print("----------------------------------------")

# ============================
# KPI 4: AVERAGE TRANSACTION VALUE
# ============================
print("Calculating Average Transaction Value per Customer...")

avg_transaction = df_clean.groupBy("customer_id") \
    .agg(avg("total_amount").alias("avg_transaction_value"))

avg_transaction.show(5)

# Save as CSV
avg_transaction.write.mode("overwrite") \
    .option("header", True) \
    .csv("data/serving/avg_transaction")

print("Average Transaction saved to
data/serving/avg_transaction")
print("----------------------------------------")

# ============================
# STOP SPARK
# ============================
spark.stop()

end_time = time.time()
execution_time = round(end_time - start_time, 2)

print("========================================")
print("   ANALYTICS LAYER COMPLETED SUCCESS   ")
print(f"   Execution Time: {execution_time} sec")
print("========================================")
```

STEP 2 — Jalankan Analytics Layer

```
source venv/bin/activate
python scripts/analytics_layer.py
```

Jika berhasil:

✓ Total revenue muncul
✓ Top 10 product muncul (Jika kategori Produk ada =>10)
✓ Revenue per category muncul

📂 **Output yang Akan Terbentuk**

```
data/
 └── serving/
     ├── total_revenue/
     ├── top_products/
     ├── category_revenue/
     └── avg_transaction/
```

Sekarang kita resmi punya:

📊 KPI layer

🟣 **BAGIAN B — VISUALIZATION LAYER**

Tujuan: Melihat bagaimana *data engineer* terhubung ke BI (*Business Intelligence*).

Kita buat 2 opsi:

🟢 **OPSI 1 (Paling Mudah) — Visualisasi dengan Pandas + Matplotlib**

Install dulu:

```
pip install pandas matplotlib
```

Buat file:

```
scripts/visualization_layer.py
```

Isi:

```
# ====================================
# VISUALIZATION LAYER
# ====================================

from pyspark.sql import SparkSession
import pandas as pd
import matplotlib.pyplot as plt

spark = SparkSession.builder \
    .appName("VisualizationLayer") \
    .master("local[*]") \
    .getOrCreate()

spark.sparkContext.setLogLevel("ERROR")

df = spark.read.parquet("data/clean/parquet/")

# Revenue per Category
```

```
category_df = df.groupBy("category") \
    .sum("total_amount") \
    .toPandas()

category_df = category_df.sort_values("sum(total_amount)",
ascending=False)

plt.figure(figsize=(8,5))
plt.bar(category_df["category"],
category_df["sum(total_amount)"])
plt.xticks(rotation=45)
plt.title("Revenue per Category")
plt.ylabel("Total Revenue")
plt.tight_layout()
plt.savefig("reports/category_revenue.png")

print("Visualization saved to reports/category_revenue.png")

spark.stop()
```

Jalankan:

```
python scripts/visualization_layer.py
```

File akan tersimpan di:

```
reports/category_revenue.png
```

Bisa buka via:

```
explorer.exe reports
```

Sekarang terlihat grafik revenue.

OPSI 2 (Lebih Enterprise) — Power BI

STEP 1 — Buka Power BI Desktop

Install jika belum tersedia:

```
https://powerbi.microsoft.com/desktop
```

Kemudian buka **Power BI Desktop**.

STEP 2 — Import Dataset Total Revenue

Klik:

```
Home → Get Data
```

Pilih:

```
Text / CSV
```

Masuk ke folder:

```
data/serving/total_revenue
```

Pilih file:

`part-00000-xxxxx.csv`

Klik:

`Load`

STEP 3 — Membuat KPI Card

Pada panel **Visualizations** klik ikon:

`Card`

Kemudian drag field:

`total_revenue`

ke area **Fields**.

Dashboard akan menampilkan angka besar:

`Rp284,070,070`

STEP 4 — Format KPI (*Key Performance Indicator*)

Klik visual KPI.

Atur pada panel **Format**:

Title:

`Total Revenue`

Font size:

`40 – 50`

Display units:

`None`

STEP 5 — Import Dataset Top Products

Klik:

`Home → Get Data → Text/CSV`

Masuk ke folder:

`data/serving/top_products`

Load file CSV.

STEP 6 — Membuat Visualisasi Top Product

Klik visual:

`Clustered Bar Chart`

Drag field:

`product`

ke:

`Y-axis`

Drag field:

`total_quantity`

ke:

`X-axis`

Urutkan:

`Sort by total_quantity → Descending`

STEP 7 — Import Dataset Revenue per Category

Klik:

`Home → Get Data → Text/CSV`

Pilih folder:

`data/serving/category_revenue`

Load file CSV.

STEP 8 — Membuat Revenue per Category Chart

Klik visual:

`Clustered Bar Chart`

Drag:

`category`

ke:

`Y-axis`

Drag:

`category_revenue`

ke:

`X-axis`

STEP 9 — Menambahkan Judul Dashboard

Klik:

`Insert → Text box`

Tulis (Title):

`E-Commerce Sales Dashboard`

Tulis (SubTitle):

`Batch Analytics – Big Data Technology`

Letakkan di bagian atas dashboard.

STEP 10 — Menyusun Layout Dashboard

Atur posisi visual menjadi:

E-Commerce Sales Dashboard

Total Revenue

Top Product | Revenue per Category

STEP 11 — Simpan Dashboard

Klik:

`File → Save`

Nama file:

`bigdata_dashboard.pbix`

5. CHECKLIST VALIDASI PRAKTIKUM 3

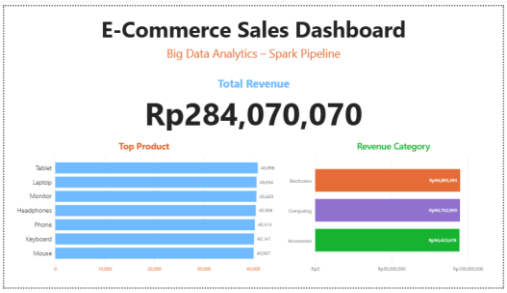
Berhasil jika:

No	Validasi	Status
1	Analytics layer berhasil dijalankan	✓
2	Folder data/serving terbentuk	✓
3	Dataset berhasil diimport ke Power BI	✓
4	KPI Total Revenue tampil	✓
5	Top Product chart muncul	✓
6	Revenue per Category chart muncul	✓
7	Dashboard memiliki judul	✓

6. OUTPUT WAJIB YANG DIKUMPULKAN

Wajib mengumpulkan:

1. Screenshot dashboard Power BI



2. File Power BI

   `bigdata_dashboard.pbix`

3. Screenshot folder serving dataset

   `data/serving`

4. Screenshot terminal saat menjalankan:

   `python scripts/analytics_layer.py`

7. RUBRIK PENILAIAN (SKOR 1–4)

Aspek	Skor 1	Skor 2	Skor 3	Skor 4
Menjalankan pipeline	Gagal	Sebagian	Berhasil	Sangat lancar
Import dataset	Tidak berhasil	Sebagian	Berhasil	Optimal
KPI visualization	Tidak ada	Kurang jelas	Baik	Sangat jelas
Chart analytics	Tidak ada	Sebagian	Lengkap	Sangat informatif
Layout dashboard	Berantakan	Kurang rapi	Rapi	Profesional

8. INSIGHT

Melalui praktikum ini dapat dipahami bahwa:

- Power BI tidak memproses Big Data
- Power BI hanya menampilkan hasil analisis

Proses utama terjadi pada:

Spark Processing Layer

Visualization Layer hanya menyajikan:

- KPI (*Key Performance Indicator*)
- Ranking
- Kategori Revenue

untuk membantu **pengambilan keputusan bisnis**.